

ANAVANDI 2026 --- Welfare Saathi

Technical Review, Logic, Workflow, Architecture & Direction

Project: Welfare Saathi\

Challenge: PS-07 --- Welfare Entitlement Assistant for Plantation
and Fishing Families\

Event: ANAVANDI Hackathon 2026\

Purpose: Review preparation / technical explanation / judge
discussion

1. Project in One Sentence

****Welfare Saathi is an offline-first, bilingual welfare entitlement assistant that collects only the household information needed for screening, evaluates that information against structured scheme rules, and explains potentially eligible schemes, missing information, required documents, and application guidance.****

The system should never guarantee that a person will receive a benefit.

The correct terminology is **"Potentially Eligible."**

2. The Problem

The challenge is not simply finding welfare schemes.

The real problem is that eligible families may not know:

- Which welfare schemes apply to them
- Which household details matter
- What documents are required
- Where they need to apply
- What to do when information is incomplete
- Why a particular scheme may or may not match them

The application therefore acts as an **access and understanding layer** between verified scheme rules and households.

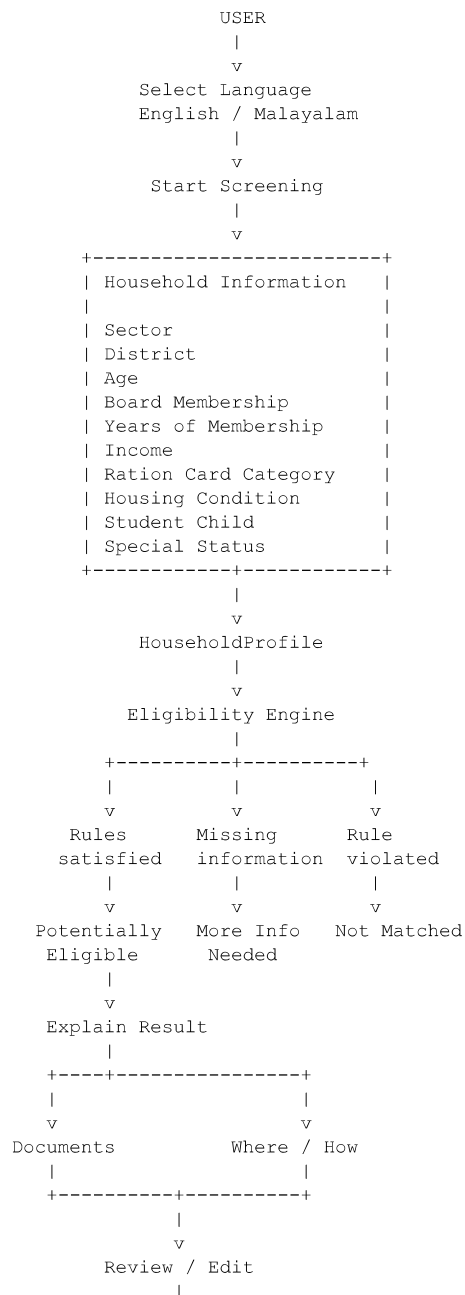
3. Core Product Idea

Welfare Saathi converts complicated scheme eligibility criteria into a simple screening workflow.

The user:

- 1. Selects a language.
- 2. Starts a screening.
- 3. Enters household information.
- 4. The system creates a structured household profile.
- 5. The eligibility engine evaluates the profile against scheme rules.
- 6. Each scheme receives an explainable result.
- 7. The user can inspect why a scheme matched.
- 8. The user can see required documents and application information.
- 9. The user can review and correct answers.
- 10. The screening can be recalculated.

4. High-Level Workflow



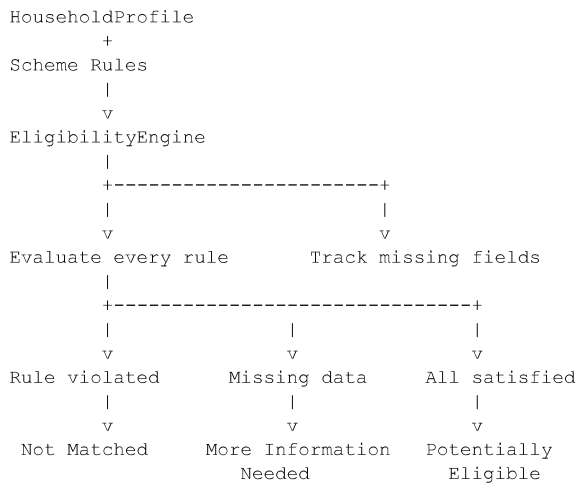
5. Core Eligibility Logic

The most important technical principle is:

*****Unknown information must never be converted into a guessed answer.*****

For every scheme, the engine evaluates its rules against the household profile.

Conceptually:



Three Result States

A. Potentially Eligible

All required rules are satisfied.

Example:

Occupation = Fishing	✓
Age = 62	✓
Board Member = Yes	✓
Income = within limit	✓
Membership = sufficient	✓

Result:

Potentially Eligible

B. More Information Needed

No rule has definitely failed, but required information is missing.

Example:

Occupation = Fishing	✓
Age = 62	✓
Board Member = Yes	✓
Income = Unknown	?

The system must not assume an income.

Result:

More Information Needed

The user should be told which information is missing.

C. Not Matched

At least one required condition is definitely violated.

Example:

Occupation = Fishing	✓
Age = 45	✗
Required age >= 60	

Result:

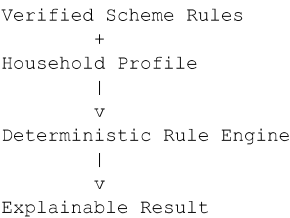
Not Matched

6. Why the Eligibility Engine Is Deterministic

The eligibility decision should not depend on an LLM.

The project architecture intentionally keeps the eligibility calculation in a pure Dart rule engine.

The important principle is:



This provides:

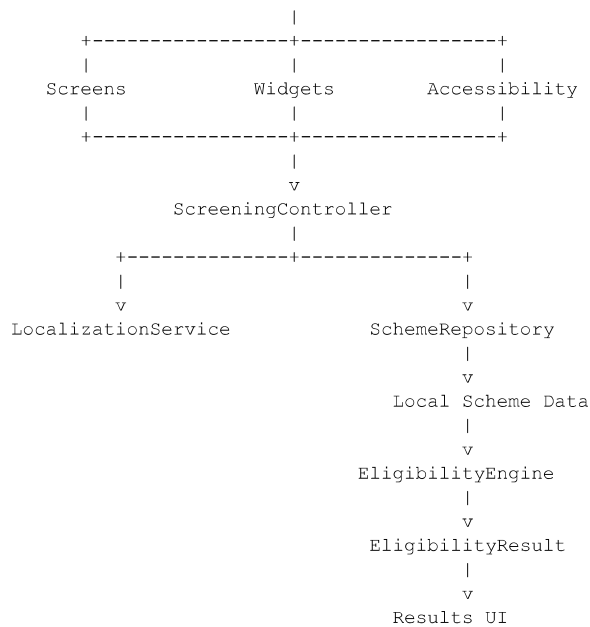
- Reproducibility
- Auditability
- Predictability
- Easier testing
- Easier debugging
- Reduced hallucination risk

Judge answer

"We deliberately kept AI out of the eligibility decision loop. Welfare eligibility is a high-consequence decision, so the result needs to be deterministic and auditable. AI can assist the interaction layer, but the verified rules remain the source of truth."

7. Architecture

The project follows a **Controller--Service--Repository--Engine** style architecture.



8. Main Architectural Layers

8.1 Presentation Layer

Responsible for:

- Welcome screen
- Screening screen
- Results
- Scheme details
- Review/edit
- Scheme directory
- Language selector
- Disclaimer
- Accessibility controls

The UI should collect and display information.

The UI should not contain the eligibility decision logic.

8.2 Controller Layer

ScreeningController

Responsible for:

- Current household profile
- Updating answers
- Triggering screening
- Holding current results
- Notifying the UI of state changes

Conceptually:

```
User Input
  |
  v
ScreeningController
  |
  v
HouseholdProfile
  |
  v
EligibilityEngine
```

8.3 Localization Service

LocalizationService

Responsible for:

- English text
- Malayalam text
- Language switching
- User-facing terminology

The system should treat Malayalam as a core accessibility feature rather than merely translating the final result.

8.4 Repository Layer

SchemeRepository

Responsible for loading scheme information from local assets.

Typical data includes:

```
schemes.json
akshaya_centres.json
sample_households.json
terminology.json
```

This separates scheme data from the evaluation engine.

8.5 Eligibility Engine

EligibilityEngine

The core decision component.

Input:

```
HouseholdProfile
```

and:

```
Scheme
```

Output:

```
EligibilityResult
```

The engine interprets structured rules using operators such as:

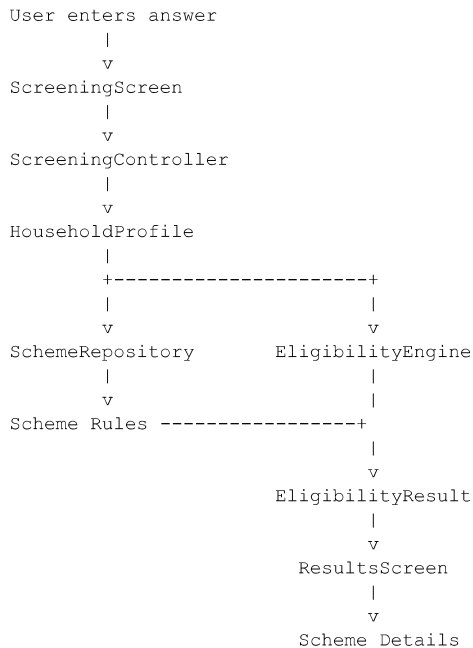
```
EQUALS
GREATER_EQUAL
```

```
LESS_EQUAL
IN_LIST
BOOLEAN
```

This means the application can add or modify scheme rules without creating a completely new evaluation algorithm for each scheme.

9. Data Flow

A complete screening flow looks like:



10. Fresh Screening Logic

A fresh screening must not contain artificial household answers.

The current implementation specifically removed silent defaults such as:

```
Age = 58
Years of Membership = 6
Board Member = true
Housing = dilapidated
```

Instead, fresh screening starts with unanswered/null values.

Examples:

```
Age           = null
Income        = null
Board membership = null
Housing condition = null
District      = null
Ration category = null
Student child  = null
```

This distinction is important:

```
Demo Profile
!=
Fresh User Screening
```

Benchmark profiles may contain intentional sample values, but a new screening must not invent user information.

11. Clearing an Answer

The user must also be able to remove an answer.

Example:

```
Age = 62
  |
  v
User clears field
  |
  v
Age = null
```

It must **not** revert to:

```
58
```

The updated `HouseholdProfile.copyWith()` implementation supports explicit clearing of fields using clear flags.

12. Review and Re-screen Workflow

The application supports correction.

```
Initial Screening
  |
  v
Results
  |
  v
Review / Edit
  |
  v
Change Answer
  |
  v
Recalculate
  |
  v
Updated Results
```

Example:

```
Income = Unknown
  |
  v
More Information Needed
  |
  v
User enters valid income
  |
  v
Re-screen
  |
  v
Potentially Eligible
```

This makes the application interactive rather than a one-time questionnaire.

13. Explainable Results

A result should not simply say:

```
Eligible
```

Instead:

Potentially Eligible

Why?

- ✓ Occupation requirement satisfied
- ✓ Age requirement satisfied
- ✓ Membership requirement satisfied
- ✓ Income requirement satisfied

Required Documents:

- Document A
- Document B

Where to Apply:

- Relevant application channel

Next Steps:

1. Prepare documents
2. Submit through the indicated channel
3. Authority verifies final eligibility

The system should always preserve the distinction between:

screening result

and

final government approval.

14. Privacy Model

The prototype follows data minimization.

The core screening does not require:

Name
Phone number
Aadhaar number
Password
Street address

Instead it uses only attributes required by the scheme rules.

The principle is:

*****Collect the minimum information required for screening.*****

The current architecture is primarily in-memory rather than building an unnecessary personal-data database.

15. Why Offline-First

The core screening can be structured around locally bundled scheme data and a local rule engine.

```
Browser
  |
  v
Local Scheme Data
  |
  v
Local Rule Engine
  |
  v
Local Result
```

Benefits:

- Reduced network dependency

- Faster screening
 - Better resilience
 - Lower data transmission
 - Easier demonstration
 - Better privacy posture
-

16. Why Not Use Firebase for the Eligibility Engine?

A backend is not necessary for the core screening calculation.

The application can perform:

```
Load rules
↓
Collect profile
↓
Evaluate locally
↓
Show result
```

This keeps the most important business logic simple and deterministic.

If future versions require remote rule updates, analytics, or authenticated case management, those can be added separately without changing the conceptual rule engine.

17. Why Not Use Gemini for Eligibility?

Do not say:

"Gemini determines whether the user is eligible."

Instead say:

"Eligibility is calculated using deterministic rules."

If AI is introduced later, a safer architecture is:

```
User
|
v
AI-assisted interaction
|
v
Structured HouseholdProfile
|
v
Deterministic Eligibility Engine
|
v
Explainable Result
```

AI should not be allowed to invent statutory eligibility criteria.

18. Scheme Data Architecture

The scheme definition should contain structured information such as:

```
Scheme
  ■■■ Name
  ■■■ Target group
  ■■■ Rules
```

- ■■■ Field
- ■■■ Operator
- ■■■ Expected value
- Required documents
- Application information
- Source/reference

The advantage is:

```

New Scheme
  |
  v
Add verified structured data
  |
  v
Same EligibilityEngine

```

rather than:

```

New Scheme
  |
  v
Write new custom Dart algorithm

```

19. Main Technical Strengths

1. Deterministic

Same input + same rules = same result.

2. Explainable

The user can understand why a scheme matched.

3. No Guessing

Unknown information remains unknown.

4. Offline-Oriented

Core screening does not need an AI API or backend decision service.

5. Data-Driven

Scheme rules are separated from the engine.

6. Bilingual

English and Malayalam are treated as first-class UI languages.

7. Correctable

Users can review answers and re-screen.

8. Privacy-Conscious

The screening does not require unnecessary identity information.

20. Main Risks to Watch

Risk 1 --- Unverified Scheme Claims

Do not claim that a scheme is officially verified unless the organiser-provided resources or source documents support that statement.

Use:

"Organiser-supplied verified scheme rules"

when that is actually the source.

Risk 2 --- Overclaiming Accessibility

Do not casually claim:

"WCAG compliant"

unless a formal accessibility evaluation supports that claim.

Instead describe the actual features implemented, such as:

- Malayalam
 - High contrast
 - Large text
 - Clear labels
-

Risk 3 --- Eligibility Guarantee

Never say:

"You are eligible."

Say:

"Potentially Eligible based on the supplied screening rules."

Final approval remains with the responsible authority.

Risk 4 --- AI Hallucination

Do not put an LLM in the core eligibility decision path.

Risk 5 --- Demo Data vs Real User Data

Clearly distinguish:

Sample / benchmark household

from:

Fresh screening

21. What NOT to Change Before Review

Unless a real bug is found, avoid unnecessary changes to:

- EligibilityEngine
- Rule operators
- Scheme data structure
- Existing tests
- Localization architecture
- Offline asset approach
- ScreeningController
- Existing benchmark profiles

Avoid major refactoring immediately before judging.

22. What Should Be Tested Before the Review

Test 1 --- Valid Potentially Eligible Case

Enter a realistic household profile that satisfies the relevant scheme rules.

Expected:

```
Potentially Eligible
```

Test 2 --- Failed Rule

Enter a value that clearly violates a required rule.

Expected:

```
Not Matched
```

Test 3 --- Missing Information

Leave a required field blank.

Expected:

```
More Information Needed
```

Test 4 --- Correction

```
Invalid / insufficient answer
↓
Not Matched / More Information Needed
↓
Edit answer
↓
Re-screen
↓
Updated result
```

Test 5 --- Fresh Screening

Start a completely new screening.

Verify there are no artificial values such as:

```
Age = 58
Membership = 6
Board Member = Yes
Housing = Dilapidated
```

Test 6 --- Clear Existing Answer

```
Age = 62
  ↓
Clear
  ↓
Age = null
```

It must not revert to a demo value.

23. Judge Question: "Explain Your Architecture"

Suggested answer

"Welfare Saathi follows a Controller--Service--Repository--Engine architecture. The Flutter UI collects household information through the screening workflow. The ScreeningController maintains the current HouseholdProfile and triggers evaluation. The SchemeRepository loads the supplied scheme data, while the LocalizationService handles English and Malayalam. The EligibilityEngine is a deterministic Dart component that evaluates the household profile against structured scheme rules. It produces Potentially Eligible, More Information Needed, or Not Matched results, which are then presented with explanations, documents and application guidance."

24. Judge Question: "How Does Eligibility Work?"

Suggested answer

"Each scheme is represented using structured rules. When the user completes screening, their answers form a HouseholdProfile. The engine evaluates every rule. If a rule is definitely violated, the scheme is Not Matched. If no rule is violated but required information is missing, it becomes More Information Needed. Only when all required rules are satisfied do we return Potentially Eligible. Unknown information is never guessed."

25. Judge Question: "Why Didn't You Use AI?"

Suggested answer

"We deliberately kept AI out of the eligibility decision loop. Welfare screening needs deterministic and explainable results. An AI model can assist with interaction or accessibility, but it should not invent or reinterpret the supplied eligibility rules. Our rule engine produces reproducible results from structured rules."

26. Judge Question: "What Happens When Information Is Missing?"

Suggested answer

"We don't treat missing information as either Yes or No. We keep it as unknown. If the missing field is required by a scheme, the system returns More Information Needed and tells the user what information is missing."

27. Judge Question: "Can You Guarantee the Benefit?"

Suggested answer

"No. The system only performs a screening against the supplied rules. We deliberately use the term Potentially Eligible because final eligibility and approval belong to the relevant authority."

28. Judge Question: "Why Do You Need a Rule Engine?"

Suggested answer

"Because we don't want to hard-code every scheme into separate UI logic. The scheme criteria are represented as structured rules, and one generic engine can evaluate those rules. This makes the system easier to test, explain and extend."

29. Judge Question: "What Happens When Rules Change?"

Suggested answer

"The rule data is separated from the engine. When verified criteria change, the structured scheme data can be updated while the generic evaluation engine remains the same, subject to validation and

testing."

30. Judge Question: "What Happens If the User Enters Wrong Information?"

Suggested answer

"The system cannot independently verify every household statement. It transparently evaluates the information provided, clearly labels the result as Potentially Eligible rather than guaranteed eligibility, and allows the user to review and correct their answers."

31. Judge Question: "Can This Scale to More Schemes?"

Suggested answer

"The architecture is data-driven. A new scheme can be represented as structured rules and supporting information, allowing the existing engine to evaluate it without creating a completely separate algorithm."

32. Judge Question: "Why Not Store Everything in a Database?"

Suggested answer

"The prototype does not need persistent personal data for its core screening workflow. Keeping the screening state local reduces unnecessary data collection and makes the core experience more resilient."

33. Future Direction

Phase 1 --- Current Prototype

Bilingual
+
Offline-oriented
+
Deterministic
+
Explainable
+
Rule-driven

Phase 2 --- Verified Rule Updates

Official / organiser-approved rules
↓
Versioned dataset

↓
Validated application update

Phase 3 --- Wider Coverage

More schemes
+
More districts
+
More beneficiary categories

Phase 4 --- Assisted Accessibility

Potential future flow:

Malayalam Voice
↓
AI-assisted extraction
↓
Structured HouseholdProfile
↓
Deterministic EligibilityEngine
↓
Explainable Result

The principle remains:

Make interaction increasingly intelligent while keeping the eligibility decision deterministic.

34. 60-Second Architecture Pitch

"Welfare Saathi uses a Controller--Service--Repository--Engine architecture. The Flutter interface collects household information and creates a structured HouseholdProfile. The ScreeningController manages the screening session. The SchemeRepository loads the supplied scheme data, while the LocalizationService supports English and Malayalam. The EligibilityEngine is a deterministic Dart rule engine that evaluates every scheme against the household profile. It produces three outcomes: Potentially Eligible, More Information Needed, or Not Matched. The results are explainable, showing why a scheme matched, what information is missing, and what documents or application steps are required. We deliberately keep AI out of the eligibility decision because the decision needs to be deterministic and auditable."

35. 60-Second Logic Pitch

"Our eligibility logic is rule-based rather than AI-generated. Each scheme contains structured rules with a field, operator and expected value. The engine evaluates every rule against the household profile. A definite rule violation produces Not Matched. If no rule is violated

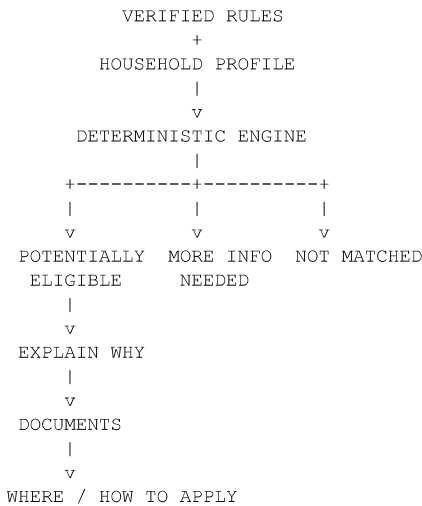
but required information is missing, the result is More Information Needed. If all required rules are satisfied, the result is Potentially Eligible. Most importantly, unknown information is never converted into a guessed answer."

36. 45-Second Workflow Pitch

"The user selects English or Malayalam, starts a screening and enters the household details required for welfare screening. These answers become a HouseholdProfile. The EligibilityEngine compares that profile with the available scheme rules and generates explainable results. The user can see why a scheme matched, what documents are needed and where to apply. If information is missing, the system asks for it rather than guessing. The user can also review and correct answers and run the screening again."

37. Core Message to Remember

The whole product can be explained with this:



The central design principle

"Keep the eligibility decision deterministic; make the experience accessible, explainable and increasingly intelligent."

38. Important Hackathon Preparation Note

The ANAVANDI participant rulebook states that teams must understand their AI-assisted work, declare AI tools, and be prepared for judges to ask either member to explain, modify or debug the solution. It also

states that the final hackathon build must be created during the official coding window and that the selection prototype/code cannot simply be reused for the main hackathon. Therefore, this document is intended as a **review and understanding guide**, not as a substitute for understanding the code you submit.

39. Final Review Checklist

Before the review, make sure both team members can explain:

- ☐ Problem
 - ☐ Target users
 - ☐ User workflow
 - ☐ HouseholdProfile
 - ☐ ScreeningController
 - ☐ SchemeRepository
 - ☐ EligibilityEngine
 - ☐ Rule operators
 - ☐ Three result states
 - ☐ Missing-information behavior
 - ☐ Fresh-screening null behavior
 - ☐ Review/re-screen flow
 - ☐ Malayalam localization
 - ☐ Privacy approach
 - ☐ Offline approach
 - ☐ Why AI is not in eligibility
 - ☐ Scheme data structure
 - ☐ Testing strategy
 - ☐ Known limitations
 - ☐ Future direction
-

40. Final Direction

Do not turn the review into a discussion about how many technologies were used.

The strongest story is:

```
Real problem
↓
Simple user input
↓
Structured household profile
↓
Verified scheme rules
↓
```

Deterministic evaluation
↓
Explainable result
↓
Documents + application guidance
↓
Review + correction

The technical quality comes from ****making the decision trustworthy, explainable and maintainable****, not from adding unnecessary AI or infrastructure.